FACULTÉ DES SCIENCES ET TECHNIQUES DE TANGER

Cycle ingénieur LSI

Atelier JEE 2

Application web e-commerce basée sur MVC2 et JPA

Réalisé par Sara Ouhaddou

Encadrant Pr. Lotfi Elaachak

Module Java EE (Servlets, JSP/JSTL, JPA)

Dépôt `jee-ecommerce-jpa-mvc2`

Date Novembre 2026

Résumé

Ce rapport présente la réalisation d'« Atlas Boutique », une application web de commerce électronique construite selon le patron MVC2 (servlets contrôleurs, vues JSP/JSTL) avec une couche de persistance JPA (EclipseLink) sur MySQL. Le périmètre retenu couvre l'internaute (client ou administrateur), la vitrine, les catégories, les produits et le panier ; la commande est volontairement laissée en perspective.

Le document explique les décisions plutôt que le code : héritage `SINGLE_TABLE`, verrouillage optimiste, règles métier portées par l'entité `Panier`, transactions centralisées dans un seul composant, injection CDI sur Tomcat, protection contre les failles courantes (CSRF, XSS, injection SQL, fixation de session) et vérification par 112 contrôles automatisés complétés d'un parcours manuel. Un épisode technique (incompatibilité entre EclipseLink 2.7.5 et les classes compilées pour Java 17) est détaillé car il illustre l'importance de lire les journaux au démarrage.

Mots-clés : Java EE, MVC2, servlets, JSP, JSTL, JPA, EclipseLink, CDI, Weld, MySQL, Tomcat, sécurité web.

Table des matières

Résumé **1**

1 Introduction **7**

1.1 Contexte et objectif 7

1.2 Périmètre retenu 7

1.3 Démarche 7

1.4 Taille du projet 8

1.5 Organisation du rapport 8

2 Conception **9**

2.1 Acteurs et besoins 9

2.2 Diagramme de classes 9

2.3 Règles métier 11

2.4 Choix de la stratégie d'héritage 11

3 Environnement et projet Maven **12**

3.1 Environnement de travail 12

3.2 Corrections apportées au `pom.xml` du sujet 12

3.3 Incident : EclipseLink et Java 17 13

4 Couche de persistance **14**

4.1 Les entités et leurs correspondances 14

4.2 L'unité de persistance 15

4.3 Création du schéma et données de test 15

4.4 Vérification dans MySQL 16

4.5 Les contraintes refusent bien les données incorrectes 19

5 Services métier et injection CDI **20**

5.1 Pourquoi une couche de services 20

5.2 Transactions centralisées 20

5.3 Règles métier : à un seul endroit 21

5.4 Validation des saisies 21

5.5 Requêtes . . . 21
5.6 Injection de dépendances avec CDI . . . 22
5.6.1 Pourquoi CDI sur Tomcat . . . 22
5.6.2 Mise en place . . . 22
5.6.3 La fabrique d'EntityManager . . . 22

6 Couche web : contrôleurs MVC2 24
6.1 Le patron MVC2 appliqué . . . 24
6.2 Classe de base . . . 24
6.3 Redirection après POST et messages éphémères . . . 24
6.4 Filtres . . . 25
6.5 Réponses AJAX . . . 25

7 Vues JSP/JSTL et interface 26
7.1 Principes . . . 26
7.2 Gabarits (tag files) . . . 26
7.3 Les pages . . . 26
7.4 Formulaires et validation . . . 28
7.5 Mini-panier en jQuery . . . 28
7.6 Administration . . . 28
7.7 Adaptation aux petits écrans . . . 31

8 Sécurité 32
8.1 Jeton CSRF . . . 32
8.2 Mots de passe . . . 33
8.3 En-têtes et politique CSP . . . 33
8.4 Ce que la sécurité ne couvre pas . . . 33

9 Tests et validation 34
9.1 Stratégie . . . 34
9.2 Contrôles automatiques . . . 34
9.3 Parcours manuel . . . 35
9.4 Ce que les tests ont révélé . . . 35

10 Gestion de version 37
10.1 Principes . . . 37
10.2 Historique . . . 37

11 Limites et perspectives **38**

11.1 Limites connues 38

11.2 Ajouter la commande 38

11.3 Autres pistes 39

12 Conclusion **40**

Bibliographie **41**

A Arborescence du projet **42**

B Comptes et données de démonstration **43**

Table des figures

2.1 Diagramme de classes de la version A (panier, vitrine, internaute) 10

4.1 Le fichier persistence.xml ouvert dans Eclipse (onglet Source) 16
4.2 Sortie de SchemaTool : contraintes appliquées, données chargées, bilan 17
4.3 Les six tables créées 17
4.4 Structure des tables produit et internaute 18
4.5 Définition de ligne_panier : clé unique, cascade et contraintes CHECK 18
4.6 Les internautes de démonstration et leur rôle 18
4.7 Les douze produits de démonstration 19
4.8 Les trois refus et leurs codes d'erreur 19

7.1 La vitrine : filtre par catégorie, recherche et grille de produits 27
7.2 Fiche produit avec étiquette de stock et choix de la quantité 27
7.3 Inscription refusée : messages sous les champs invalides 28
7.4 Ajout au panier sans rechargement : tiroir latéral, badge et notification 29
7.5 Page du panier : lignes, quantités, total et récapitulatif 29
7.6 Refus d'une quantité supérieure au stock disponible 30
7.7 Administration : liste des produits et actions 30
7.8 Affichage sur petit écran 31

9.1 Résultat de la classe Verifications 35
9.2 Contrôle dans MySQL après une action : la ligne de panier correspond à ce qui est affiché 35

Liste des tableaux

2.1 Acteurs et fonctionnalités 9
2.2 Règles métier et double protection 11
2.3 Stratégies d'héritage JPA 11

3.1 Versions utilisées 12

4.1 Décisions de mapping 14
4.2 Écritures refusées par la base 19

5.1 Interfaces de service 20

6.1 Contrôleurs, adresses et actions 24

7.1 Gabarits JSP 26

8.1 Menaces et protections 32

9.1 Classes de contrôle 34
9.2 Scénarios de test manuels 36

10.1 Séquence de commits 37

B.1 Comptes de démonstration 43

CHAPITRE 1

Introduction

1.1 Contexte et objectif

Cet atelier s'inscrit dans le module Java EE du cycle ingénieur LSI de la FST de Tanger, sous la direction de Pr. Lotfi Elaachak. Le sujet demande de construire une application web de commerce électronique en respectant deux contraintes d'architecture : le patron **MVC2**, où les servlets jouent le rôle de contrôleurs et les pages JSP celui de vues, et la persistance par **JPA**, ici avec EclipseLink, au lieu de requêtes JDBC écrites à la main comme à l'atelier 1.

L'application, baptisée *Atlas Boutique*, reprend l'identité visuelle de l'atelier 1 (barre latérale sombre, vert profond, safran pour l'élément courant) avec une interface plus soignée. Elle fonctionne sans aucune dépendance externe au moment de l'exécution : la bibliothèque jQuery est livrée avec l'application, il n'y a ni CDN ni police distante.

1.2 Périmètre retenu

Le sujet cite un grand nombre de classes. Il recommande lui-même de se concentrer sur le panier, la vitrine et l'internaute. J'ai donc retenu la « version A » du périmètre :

— l'**internaute**, spécialisé en **client** et **administrateur** ;

— la **vitrine** et ses **catégories** ;

— les **produits** ;

— le **panier** et ses **lignes**.

La **commande** n'est pas réalisée. Ce choix est assumé : une commande bien faite exige de décrémenter le stock dans une transaction avec contrôle de concurrence, ce qui mérite un atelier à part entière. Le chapitre 11 décrit comment elle s'ajouterait sans remettre en cause l'existant.

1.3 Démarche

Trois principes ont guidé le travail.

1. **Décider, puis justifier.** Chaque choix structurant (stratégie d'héritage, verrouillage, gestion des transactions) est expliqué par son pourquoi et comparé à au moins une alternative.
2. **Vérifier à chaque étape.** Chaque couche a été validée avant de passer à la suivante : tables et contraintes dans MySQL, contrôles automatiques, puis parcours dans le navigateur.

3. **Sécurité par défaut.** Les protections (jeton CSRF, échappement, requêtes paramétrées, en-têtes) font partie de la structure de l'application, elles ne sont pas ajoutées à la fin.

1.4 Taille du projet

Le projet compte environ 4 000 lignes de Java pour le code de production (45 classes), 500 lignes pour les tests (7 classes), 830 lignes pour les vues et les gabarits JSP, et 580 lignes de CSS et de JavaScript.

1.5 Organisation du rapport

Le chapitre 2 présente la conception (classes, règles métier, choix d'héritage). Le chapitre 3 décrit l'environnement et le projet Maven. Les chapitres 4 et 5 traitent de la persistance et des services. Les chapitres 6 et 7 détaillent la couche web et les vues. La sécurité fait l'objet du chapitre 8, les tests du chapitre 9 et la gestion de version du chapitre 10. Le rapport se termine par les limites, les perspectives et une conclusion.

CHAPITRE 2

Conception

2.1 Acteurs et besoins

Trois profils utilisent l'application. Le tableau 2.1 résume ce que chacun peut faire.

Table 2.1. Acteurs et fonctionnalités

Acteur	Fonctionnalités
Visiteur	Parcourir la vitrine, filtrer par catégorie, rechercher, ouvrir la fiche d'un produit, créer un compte, se connecter.
Client	Tout ce que fait le visiteur, plus : ajouter un produit au panier, modifier une quantité, retirer une ligne, vider le panier.
Administrateur	Créer, modifier, retirer de la vente et supprimer des produits ; gérer les catégories.

2.2 Diagramme de classes

La figure 2.1 (page suivante) donne le modèle du domaine. Les relations suivent la logique du sujet.

— **Héritage** : `Client` et `Administrateur` spécialisent la classe abstraite `Internaute`.

— **Client** – **Panier**, 1 à 1 : un client possède un seul panier, créé en même temps que lui. Un administrateur n'en a pas.

— **Panier** – **LignePanier**, composition : une ligne n'existe que dans son panier ; supprimer le panier supprime ses lignes.

— **LignePanier** – **Produit**, association : la ligne référence un produit mais ne le possède pas.

— **Vitrine** – **Catégorie** – **Produit**, agrégations : la vitrine présente des catégories, qui regroupent des produits.

— **Administrateur** – **Produit**, dépendance : l'administrateur gère le catalogue, sans lien persistant.

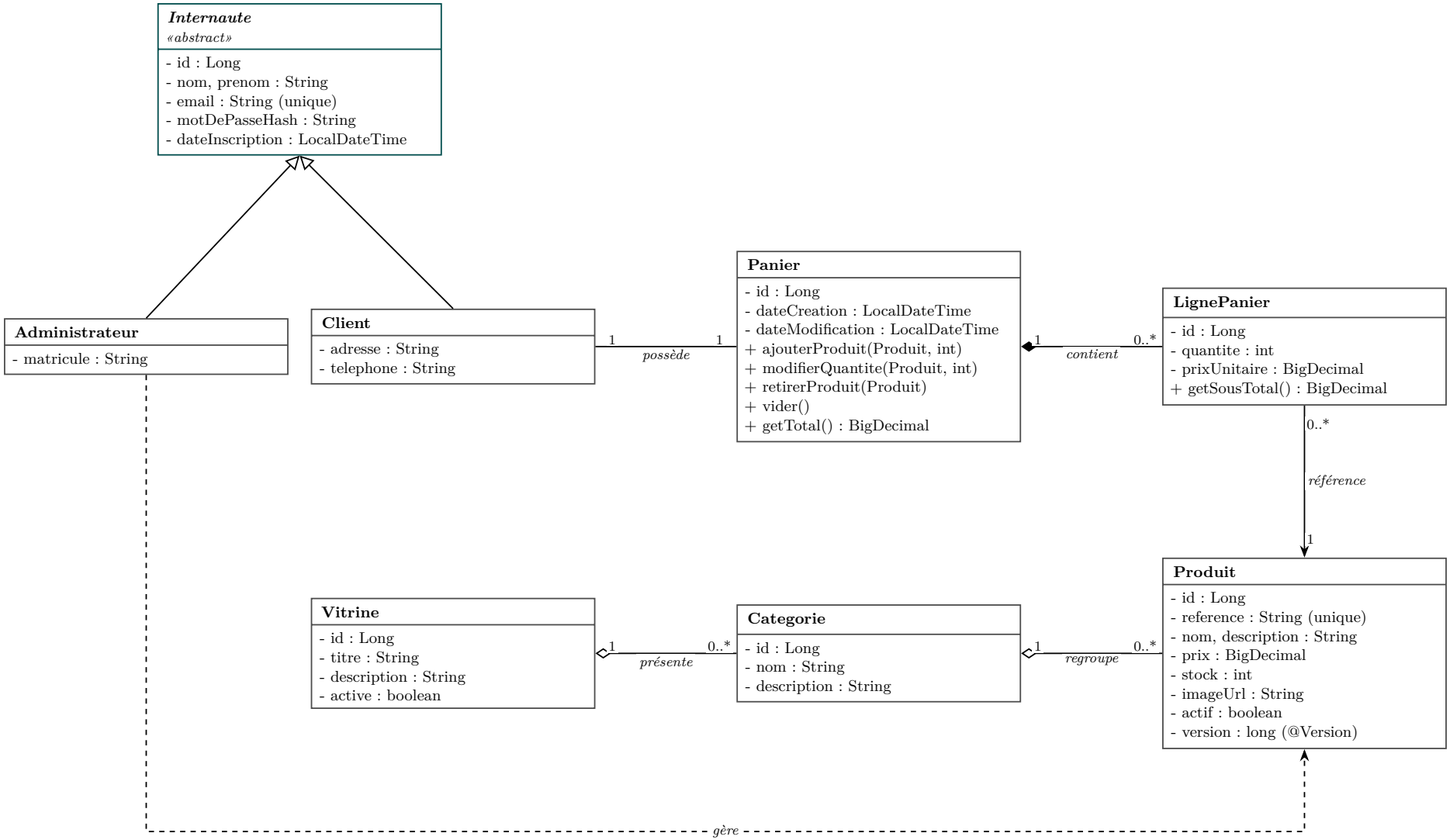


Figure 2.1. Diagramme de classes de la version A (panier, vitrine, internaute)

2.3 Règles métier

Les règles qui protègent la cohérence du panier sont écrites **dans l'entité `Panier`** et non dans les servlets ou les services. Une règle placée dans le domaine ne peut pas être oubliée par un nouvel écran : quel que soit l'appelant, la même vérification s'applique. Le tableau 2.2 les liste avec l'endroit où elles sont imposées une seconde fois, en base, quand c'est possible.

Table 2.2. Règles métier et double protection

N°	Règle	Garde-fou en base
R1	Une quantité de ligne est comprise entre 1 et 99.	`CHECK quantite >= 1`
R2	Seul un produit actif peut être ajouté au panier.	—
R3	La quantité d'une ligne ne dépasse pas le stock disponible.	`CHECK stock >= 0`
R4	Ajouter un produit déjà présent fusionne la ligne (les quantités s'additionnent).	`UNIQUE (panier, produit)`
R5	Le prix de la ligne est figé au moment de l'ajout.	`CHECK prix_unitaire >= 0`
R6	Le total est la somme des sous-totaux, calculée en `BigDecimal`.	—
R7	L'adresse électronique est unique, normalisée en minuscules.	`UNIQUE (email)`
R8	La référence d'un produit est unique, en majuscules.	`UNIQUE (reference)`
R9	Un produit présent dans un panier ne peut pas être supprimé (on le retire de la vente).	clé étrangère
R10	Une catégorie qui contient des produits ne peut pas être supprimée.	clé étrangère
R11	Un internaute a obligatoirement un rôle (`CLIENT` ou `ADMIN`).	`NOT NULL` + `CHECK`

2.4 Choix de la stratégie d'héritage

JPA propose trois façons de stocker une hiérarchie. Le tableau 2.3 les compare pour notre cas.

Table 2.3. Stratégies d'héritage JPA

Stratégie	Avantages	Inconvénients
`SINGLE_TABLE`	Une seule table, aucune jointure, authentification par une requête simple.	Colonnes propres aux sous-classes nullables ; une colonne discriminante.
`JOINED`	Modèle normalisé, pas de colonnes inutiles.	Jointure à chaque lecture, plus de tables.
`TABLE_PER_CLASS`	Chaque classe a sa table complète.	Requêtes polymorphes par `UNION`, clés à gérer entre tables.

J'ai retenu **`SINGLE_TABLE`**. La hiérarchie est petite (deux sous-classes qui ajoutent chacune un ou deux champs), et la lecture la plus fréquente est la connexion : retrouver un internaute par son adresse sans savoir à l'avance s'il est client ou administrateur. Avec une table unique, c'est une seule requête sans jointure. Le défaut de la stratégie, la colonne discriminante laissée nullable par EclipseLink, est corrigé par une contrainte `NOT NULL` et un `CHECK` (chapitre 4).

CHAPITRE 3

Environnement et projet Maven

3.1 Environnement de travail

L'environnement est celui de l'atelier 1, aucune installation supplémentaire n'a été nécessaire (tableau 3.1).

Table 3.1. Versions utilisées

Composant	Version
Système	Windows 11
JDK	Eclipse Temurin 17.0.18
Maven	3.9
Serveur d'applications	Apache Tomcat 9.0.123 (port 8081)
Base de données	MySQL 8 (`ecommerce_jpa`)
IDE	Eclipse IDE for Enterprise Java Developers
API Servlet / JSP / JSTL	4.0.1 / 2.3.3 / 1.2
JPA et fournisseur	JPA 2.2, EclipseLink 2.7.5
Injection de dépendances	CDI 2.0, Weld Servlet 3.1.9.Final
Pilote JDBC	MySQL Connector/J 8.0.33

Pourquoi Tomcat 9 et pas 10 ? EclipseLink 2.7.x utilise l'espace de noms `javax.*`. Tomcat 10 et suivants sont passés à `jakarta.*` : les deux ne sont pas compatibles sans migration. Tomcat 9 est donc le bon choix pour cette version d'EclipseLink.

3.2 Corrections apportées au `pom.xml` du sujet

Le bloc fourni dans l'énoncé ne peut pas être copié tel quel. Trois points ont été corrigés.

1. **Balises cassées** : la troisième dépendance a ses balises inversées et il manque son `</dependency>`. Le fichier ne serait pas lu par Maven.
2. **Doublon JPA** : `org.eclipse.persistence:javax.persistence:2.0.0` et `eclipselink:2.7.5` fournissent tous deux l'API `javax.persistence`, dans deux versions différentes. EclipseLink 2.7.5 embarque déjà JPA 2.2 : je garde `eclipselink` seul pour éviter un conflit de classes.
3. **Pilote MySQL** : l'énoncé cite `mysql-connector-java:8.0.12`. Par cohérence avec l'atelier 1 et avec MySQL 8, j'utilise `mysql-connector-j:8.0.33`.

Listing 3.1. Dépendances principales du pom.xml (extrait)

```
<dependency>
  <groupId>org.eclipse.persistence</groupId>
  <artifactId>eclipselink</artifactId>
  <version>2.7.5</version>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.0.33</version>
</dependency>
<dependency>
  <groupId>org.jboss.weld.servlet</groupId>
  <artifactId>weld-servlet-shaded</artifactId>
  <version>3.1.9.Final</version>
</dependency>
```

Les API Servlet, JSP, `javax.inject` et CDI sont en portée `provided` : Tomcat ou Weld les fournissent à l'exécution, il ne faut pas les embarquer dans le WAR. JSTL et le pilote MySQL, eux, sont embarqués.

3.3 Incident : EclipseLink et Java 17

Au premier lancement de l'outil de création du schéma, la console affichait :

```
[EL Severe]: metadata: ... class was compiled with an unsupported JDK.
java.lang.IllegalArgumentException: Unsupported class file major version 61
```

Diagnostic. EclipseLink 2.7.5 lit les annotations des entités avec une copie interne de la bibliothèque ASM, qui date de 2019. Cette version ne sait pas lire les fichiers `.class` produits par Java 17 (version majeure 61). Le projet était compilé avec `-release 17` ; la lecture des entités échouait, et la création des tables avec.

Solution retenue. Compiler avec `maven.compiler.release=11` (version majeure 55, lisible par cette ASM) tout en continuant à exécuter sur le JDK 17. Cela a demandé de réécrire une comparaison utilisant la syntaxe `instanceof` avec variable, introduite en Java 16. Le choix inverse, exécuter Tomcat sur un JDK 11, aurait obligé à installer un second JDK. Une mise à jour d'EclipseLink vers la série 3.x ou 4.x demanderait de quitter `javax.*`, donc de changer aussi de Tomcat.

Message résiduel. Au démarrage de Tomcat, la console contient encore des lignes `[EL Severe] ... The java.lang.Object class was compiled with an unsupported JDK`, pour `Object`, `List`, `String`, `BigDecimal`... Il s'agit des classes *du JDK lui-même*, que le même mécanisme tente de lire. Elles n'empêchent rien : la ligne suivante confirme que la connexion à la base réussit et que les entités sont chargées. Ce bruit est documenté ici plutôt que masqué, parce que le masquer cacherait aussi de vraies erreurs de métadonnées.

CHAPITRE 4

Couche de persistance

4.1 Les entités et leurs correspondances

Huit classes sont persistantes (`Internaute`, `Client`, `Administrateur`, `Vitrine`, `Categorie`, `Produit`, `Panier`, `LignePanier`). Le tableau 4.1 récapitule les décisions de mapping ; le *pourquoi* de chacune est donné ensuite.

Table 4.1. Décisions de mapping

Sujet	Décision	Raison
Accès aux données	Sur les champs (`@Id` sur l'attribut)	Les accesseurs peuvent contenir de la logique (normalisation de l'e-mail) sans être appelés par JPA.
Montants	`BigDecimal`, échelle 2	Pas d'erreur d'arrondi, contrairement à `double`.
Concurrence	`@Version` sur `Produit`	Détecte deux modifications simultanées sans verrou en base.
Client – Panier	`@OneToOne`, cascade `ALL`, `orphanRemoval`	Le panier naît et meurt avec son client.
Panier – Lignes	`@OneToMany` avec `@CascadeOnDelete`	Supprimer un panier supprime ses lignes, y compris au niveau SQL (`ON DELETE CASCADE`).
Ligne – Produit	`@ManyToOne`, sans cascade	Une ligne référence un produit, elle n'en est pas propriétaire.
Listes exposées	Copies en lecture seule	Aucun code externe ne modifie une collection sans passer par les règles métier.

Le verrouillage optimiste. Deux administrateurs peuvent ouvrir le même produit, ou un client peut ajouter au panier le dernier exemplaire pendant qu'on modifie son stock. Un verrou pessimiste bloquerait des lignes en base pendant la réflexion d'un utilisateur ; ce n'est pas acceptable dans une application web. Avec `@Version`, chaque mise à jour incrémente un numéro, et une écriture fondée sur un numéro périmé échoue. L'application traduit cet échec en message clair (« Cet élément a été modifié par quelqu'un d'autre... »). Dans les opérations de panier, le produit est lu avec `LockModeType.OPTIMISTIC` : si son numéro change avant la validation, la transaction échoue au lieu de laisser passer une quantité calculée sur un stock périmé.

Le prix figé. Une ligne de panier recopie le prix du produit au moment de l'ajout. Si l'administrateur change le prix ensuite, le panier du client ne bouge pas, ce qui évite qu'un total change sous ses yeux.

4.2 L'unité de persistance

Le fichier `persistence.xml` ne contient ni mot de passe ni adresse de base. Il décrit seulement ce qui ne varie pas d'une machine à l'autre.

Listing 4.1. persistence.xml (sans les commentaires)

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.2"
             xmlns="http://xmlns.jcp.org/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence http://xmlns.jcp.org/xml/ns/
    persistence/persistence_2_2.xsd">
  <persistence-unit name="ecommercePU" transaction-type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
    <class>ma.fstt.entities.Internaute</class>
    <class>ma.fstt.entities.Client</class>
    <class>ma.fstt.entities.Administrateur</class>
    <class>ma.fstt.entities.Vitrine</class>
    <class>ma.fstt.entities.Categorie</class>
    <class>ma.fstt.entities.Produit</class>
    <class>ma.fstt.entities.Panier</class>
    <class>ma.fstt.entities.LignePanier</class>
    <exclude-unlisted-classes>true</exclude-unlisted-classes>
    <shared-cache-mode>NONE</shared-cache-mode>
    <validation-mode>NONE</validation-mode>
    <properties>
      <property name="eclipselink.target-database" value="MySQL"/>
      <property name="eclipselink.weaving" value="false"/>
    </properties>
  </persistence-unit>
</persistence>
```

Les choix notables sont les suivants.

— `RESOURCE_LOCAL` : Tomcat n'est pas un serveur JTA, l'application gère donc ses transactions elle-même (section 5.2).

— Liste explicite des classes : l'analyse automatique du classpath n'est pas fiable sous Tomcat.

— `shared-cache-mode NONE` : pas de cache partagé, donc ce que l'on voit dans MySQL est exactement ce que l'application lit. C'est un choix de simplicité, adapté à un projet pédagogique.

— `eclipselink.weaving=false` : il n'y a pas d'agent de tissage sous Tomcat. Conséquence connue, signalée par un avertissement au démarrage : les relations `@OneToOne` ne sont pas chargées paresseusement.

Identifiants hors du dépôt. L'adresse de la base, l'utilisateur et le mot de passe viennent d'un fichier `db.properties`, ignoré par Git. Le dépôt ne contient qu'un modèle, `db.properties.example`. Une fuite d'identifiants dans un historique Git est difficile à effacer : la précaution est donc prise dès le premier commit. La classe `PersistenceConfig` lit ce fichier et construit les propriétés de la fabrique d'`EntityManager`.

4.3 Création du schéma et données de test

Le schéma n'est pas laissé au hasard du démarrage de l'application. Un outil en ligne de commande, `SchemaTool`, le crée explicitement :

1. il demande de taper `RESET` avant de supprimer et recréer les tables (option `-yes` pour les

scripts) ;

2. il applique les contraintes `CHECK` de `constraints.sql`, que JPA ne sait pas exprimer ;
3. il charge les données de démonstration : une vitrine, quatre catégories, douze produits dont un en rupture (`MAI-002`), un à trois exemplaires (`INF-003`) et un retiré de la vente (`MOD-003`), un administrateur et deux clients.

Séparer la création du schéma du démarrage de l'application évite qu'un redéploiement modifie la base. Pour la même raison, `jpa.ddl` vaut `none` lors de l'exécution normale.

persistence.xml

```
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_2.xsd (xsi:schemaLocation with catalog)
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.2"
             xmlns="http://xmlns.jcp.org/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence http://xmlns.jcp.org/xml/ns/persistence/persistence_2_2.xsd">
  <!--
    RESOURCE_LOCAL: Tomcat is not a JTA server, so the application manages its
    own transactions. JDBC URL, credentials, DDL mode and log level are NOT
    here: they come from db.properties (see PersistenceConfig).
  -->
  <persistence-unit name="ecommercePU" transaction-type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>

    <!-- Explicit list: classpath scanning is unreliable on Tomcat. -->
    <class>ma.fstt.entities.Internaute</class>
    <class>ma.fstt.entities.Client</class>
    <class>ma.fstt.entities.Administrateur</class>
    <class>ma.fstt.entities.Vitrine</class>
    <class>ma.fstt.entities.Categorie</class>
    <class>ma.fstt.entities.Produit</class>
    <class>ma.fstt.entities.Panier</class>
    <class>ma.fstt.entities.LignePanier</class>
    <exclude-unlisted-classes>true</exclude-unlisted-classes>

    <!-- No shared (L2) cache: what you see in MySQL is what the application reads. -->
    <shared-cache-mode>NONE</shared-cache-mode>
    <!-- No Bean Validation provider on the classpath. -->
    <validation-mode>NONE</validation-mode>

    <properties>
      <property name="eclipselink.target-database" value="MySQL"/>
      <!-- No weaving agent on Tomcat: silences the related warnings. -->
      <property name="eclipselink.weaving" value="false"/>
    </properties>
  </persistence-unit>
</persistence>
```

Figure 4.1. Le fichier persistence.xml ouvert dans Eclipse (onglet Source)

4.4 Vérification dans MySQL

La base a été inspectée directement, sans passer par l'application, pour valider le mapping avant d'écrire la moindre servlet.

— Les six tables attendues existent (figure 4.3) : `categorie`, `internaute`, `ligne_panier`, `panier`, `produit`, `vitrine`.

— `produit.prix` est un `decimal(10,2)`, `produit.version` un `bigint`, et `reference` porte une contrainte d'unicité (figure 4.4).

— Dans `internaute`, les colonnes propres aux sous-classes (`adresse`, `telephone`, `matricule`, `panier_id`) sont nullables : c'est la contrepartie de `SINGLE_TABLE`.

— `ligne_panier` porte `ON DELETE CASCADE`, l'unicité (`panier_id`, `produit_id`) et deux `CHECK` (figure 4.5).

— Les accents sont intacts (« mécanique », « Écharpe »), ce qui valide la chaîne UTF-8 complète, de la base jusqu'à l'affichage (figures 4.6 et 4.7).

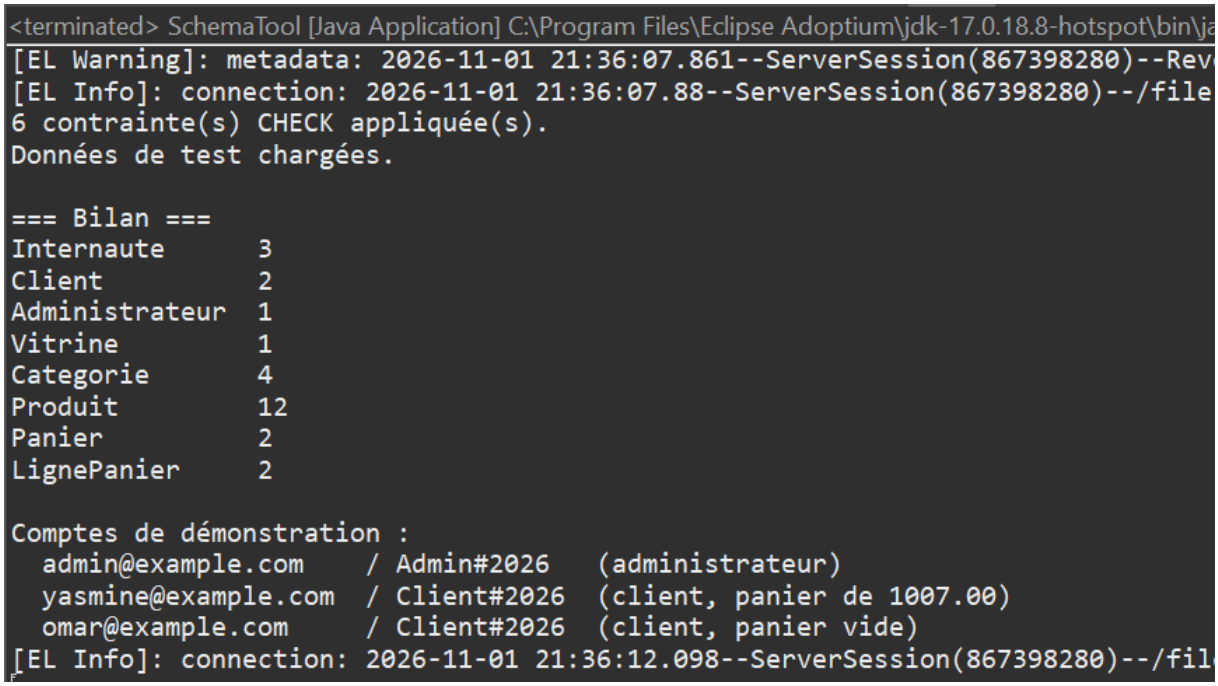


Figure 4.2. Sortie de SchemaTool : contraintes appliquées, données chargées, bilan



Figure 4.3. Les six tables créées

```
PS C:\dev\ecommerce-jpa-mvc2> mysql -u root -p ecommerce_jpa -e "DESCRIBE produit;
 DESCRIBE internaute;"
Enter password: ****************
+-------------+---------------+------+-----+---------+----------------+
| Field       | Type          | Null | Key | Default | Extra          |
+-------------+---------------+------+-----+---------+----------------+
| id          | bigint        | NO   | PRI | NULL    | auto_increment |
| actif       | tinyint(1)    | NO   |     | 0       |                |
| description | varchar(2000) | YES  |     | NULL    |                |
| image_url   | varchar(255)  | YES  |     | NULL    |                |
| nom         | varchar(150)  | NO   |     | NULL    |                |
| prix        | decimal(10,2) | NO   |     | NULL    |                |
| reference   | varchar(40)   | NO   | UNI | NULL    |                |
| stock       | int           | NO   |     | NULL    |                |
| version     | bigint        | NO   |     | NULL    |                |
| categorie_id | bigint       | NO   | MUL | NULL    |                |
+-------------+---------------+------+-----+---------+----------------+
+------------------+--------------+------+-----+---------+----------------+
| Field            | Type         | Null | Key | Default | Extra          |
+------------------+--------------+------+-----+---------+----------------+
| id               | bigint       | NO   | PRI | NULL    | auto_increment |
| type_internaute  | varchar(20)  | YES  |     | NULL    |                |
| date_inscription | datetime(3)  | NO   |     | NULL    |                |
| email            | varchar(150) | NO   | UNI | NULL    |                |
| mot_de_passe_hash | varchar(255) | NO  |     | NULL    |                |
| nom              | varchar(80)  | NO   |     | NULL    |                |
| prenom           | varchar(80)  | NO   |     | NULL    |                |
| adresse          | varchar(255) | YES  |     | NULL    |                |
| telephone        | varchar(30)  | YES  |     | NULL    |                |
| panier_id        | bigint       | YES  | UNI | NULL    |                |
| matricule        | varchar(30)  | YES  | UNI | NULL    |                |
+------------------+--------------+------+-----+---------+----------------+
```

Figure 4.4. Structure des tables produit et internaute

```
PS C:\dev\ecommerce-jpa-mvc2> mysql -u root -p ecommerce_jpa -e "SHOW CREATE TABLE ligne_panier\G"
Enter password: ****************
*************************** 1. row ***************************
       Table: ligne_panier
Create Table: CREATE TABLE `ligne_panier` (
  `id` bigint NOT NULL AUTO_INCREMENT,
  `prix_unitaire` decimal(10,2) NOT NULL,
  `quantite` int NOT NULL,
  `panier_id` bigint NOT NULL,
  `produit_id` bigint NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_ligne_panier_produit` (`panier_id`,`produit_id`),
  KEY `FK_ligne_panier_produit_id` (`produit_id`),
  CONSTRAINT `FK_ligne_panier_panier_id` FOREIGN KEY (`panier_id`) REFERENCES `panier` (`id`) ON DELETE CASCADE,
  CONSTRAINT `FK_ligne_panier_produit_id` FOREIGN KEY (`produit_id`) REFERENCES `produit` (`id`),
  CONSTRAINT `chk_ligne_prix` CHECK ((`prix_unitaire` >= 0)),
  CONSTRAINT `chk_ligne_quantite` CHECK ((`quantite` >= 1))
) ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Figure 4.5. Définition de ligne_panier : clé unique, cascade et contraintes CHECK

```
PS C:\dev\ecommerce-jpa-mvc2> mysql -u root -p ecommerce_jpa
-e "SELECT type_internaute, email, panier_id FROM internaute;
"
Enter password: ****************
+-----------------+---------------------+-----------+
| type_internaute | email               | panier_id |
+-----------------+---------------------+-----------+
| ADMIN           | admin@example.com   |      NULL |
| CLIENT          | omar@example.com    |         2 |
| CLIENT          | yasmine@example.com |         1 |
+-----------------+---------------------+-----------+
```

Figure 4.6. Les internautes de démonstration et leur rôle

```
PS C:\dev\ecommerce-jpa-mvc2> mysql -u root -p ecommerce_jpa -e "SELECT p.reference, p.nom,
p.prix, p.stock, p.actif FROM produit p ORDER BY p.reference;"
Enter password: *****************
+-----------+-----------------------------+--------+-------+-------+
| reference | nom                         | prix   | stock | actif |
+-----------+-----------------------------+--------+-------+-------+
| INF-001   | Clavier mécanique 75 %      | 549.00 |    25 |     1 |
| INF-002   | Souris sans fil ergonomique | 229.00 |    40 |     1 |
| INF-003   | SSD externe 1 To            | 899.00 |     3 |     1 |
| MAI-001   | Cafetière filtre 12 tasses  | 389.00 |    12 |     1 |
| MAI-002   | Bouilloire en verre 1,7 L   | 199.00 |     0 |     1 |
| MAI-003   | Set de 6 verres à thé       | 119.00 |    60 |     1 |
| MOD-001   | Sac à dos urbain 35 L       | 299.00 |    18 |     1 |
| MOD-002   | Casquette coton brodée      |  89.00 |    50 |     1 |
| MOD-003   | Écharpe en laine            | 159.00 |     7 |     0 |
| SPO-001   | Tapis de yoga antidérapant  | 179.00 |    30 |     1 |
| SPO-002   | Haltères réglables 2 × 10 kg| 649.00 |     5 |     1 |
| SPO-003   | Gourde isotherme 750 ml     | 139.00 |    80 |     1 |
+-----------+-----------------------------+--------+-------+-------+
```

Figure 4.7. Les douze produits de démonstration

4.5 Les contraintes refusent bien les données incorrectes

Trois écritures invalides ont été tentées à la main. La base les refuse toutes, ce qui montre qu'elle protège ses données même si une règle était oubliée dans le code Java (tableau 4.2).

Table 4.2. Écritures refusées par la base

Tentative	Code MySQL	Contrainte
Mettre un stock à −1	3819	`chk_produit_stock`
Insérer un second internaute avec la même adresse	1062	`uk_internaute_email`
Supprimer un produit présent dans un panier	1451	clé étrangère de `ligne_panier`

```
PS C:\dev\ecommerce-jpa-mvc2> mysql -u root -p ecommerce_jpa -e "UPDATE produit SET stock =
-1 WHERE reference = 'INF-001';"
Enter password: *****************
ERROR 3819 (HY000) at line 1: Check constraint 'chk_produit_stock' is violated.
PS C:\dev\ecommerce-jpa-mvc2> mysql -u root -p ecommerce_jpa -e "INSERT INTO internaute (typ
e_internaute, nom, prenom, email, mot_de_passe_hash, date_inscription) VALUES ('CLIENT','X',
'Y','yasmine@example.com','h',NOW());"
Enter password: *****************
ERROR 1062 (23000) at line 1: Duplicate entry 'yasmine@example.com' for key 'internaute.uk_i
nternaute_email'
PS C:\dev\ecommerce-jpa-mvc2> mysql -u root -p ecommerce_jpa -e "DELETE FROM produit WHERE r
eference = 'INF-001';"
Enter password: *****************
ERROR 1451 (23000) at line 1: Cannot delete or update a parent row: a foreign key constraint
 fails (`ecommerce_jpa`.`ligne_panier`, CONSTRAINT `FK_ligne_panier_produit_id` FOREIGN KEY
(`produit_id`) REFERENCES `produit` (`id`))
```

Figure 4.8. Les trois refus et leurs codes d'erreur

Une observation corrigée. La première vérification a montré que `type_internaute`, la colonne discriminante, était nullable : EclipseLink la crée ainsi. Un insertion manuelle pouvait donc créer un utilisateur sans rôle. Le fichier `constraints.sql` impose désormais `NOT NULL` et un `CHECK` limitant les valeurs à `CLIENT` et `ADMIN` (règle R11).

CHAPITRE 5

Services métier et injection CDI

5.1 Pourquoi une couche de services

Les servlets ne parlent jamais à JPA. Elles appellent quatre interfaces, qui décrivent *ce que fait* l'application sans dire *comment*. Le tableau 5.1 les présente.

Table 5.1. Interfaces de service

Interface	Responsabilité
`InternauteService`	Inscrire un client, authentifier, retrouver un internaute.
`CatalogueService`	Lecture publique : vitrine active, catégories, produits par page de 9, recherche, fiche produit.
`GestionCatalogueService`	Administration : produits et catégories (création, modification, retrait, suppression).
`PanierService`	Consulter, ajouter, modifier une quantité, retirer, vider.

Ce découpage a deux effets. Les servlets restent courtes : lire les paramètres, appeler un service, choisir la vue. Et les tests peuvent remplacer un service par une version simulée sans base de données (c'est ce que fait la classe `ServiceChecks`).

5.2 Transactions centralisées

Sans serveur JTA, chaque opération doit ouvrir un `EntityManager`, une transaction, valider ou annuler, puis fermer. Répété dans chaque méthode, ce code est une source d'oublis (une transaction jamais refermée, une connexion qui fuit). Un composant unique, `TransactionRunner`, en est responsable. Il offre deux méthodes : `read` pour les lectures, sans transaction, et `write` pour les écritures.

Listing 5.1. TransactionRunner.write : validation, annulation, fermeture

```
/** Read-write work: commit when it returns, rollback when it throws. */
public <T> T write(Function<EntityManager, T> work) {
    EntityManager em = emf.createEntityManager();
    EntityTransaction tx = em.getTransaction();
    try {
        tx.begin();
        T result = work.apply(em);
        tx.commit();
        return result;
    } catch (RuntimeException e) {
        if (tx.isActive()) {
            tx.rollback();
        }
        throw SqlErrors.traduire(e);
    } finally {
        em.close();
    }
```

```
    }
}
```

Le `finally` garantit la fermeture quoi qu'il arrive. L'exception SQL brute n'est jamais montrée à l'utilisateur : `SqlErrors.traduire` la convertit en message métier lisible. Les codes MySQL traités sont 1062 (doublon), 1451 et 1452 (clé étrangère), 1048 (valeur obligatoire manquante), 1406 (valeur trop longue), 3819 et 4025 (contrainte `CHECK`), ainsi que l'échec de verrouillage optimiste, reconnu par son nom de classe.

5.3 Règles métier : à un seul endroit

Les quantités, le stock et les produits inactifs sont contrôlés dans l'entité `Panier`, pas dans le service. Le service se contente d'orchestrer : charger le client, charger le produit, appeler la méthode du panier, valider.

Listing 5.2. Panier.ajouterProduit : fusion de lignes, prix figé

```
    /** Adds {@code quantite} units of a product; increases the line if it already exists. */
    public LignePanier ajouterProduit(Produit produit, int quantite) {
        exigerProduit(produit);
        exigerQuantiteValide(quantite);
        if (!produit.isActif()) {
            throw new MetierException("Le produit « " + produit.getNom()
                    + " » n'est plus disponible à la vente.");
        }
        LignePanier ligne = trouverLigne(produit);
        int quantiteCible = (ligne == null ? 0 : ligne.getQuantite()) + quantite;
        verifierQuantiteCible(produit, quantiteCible);

        if (ligne == null) {
            // The price is frozen here; later price changes do not alter the cart line.
            ligne = new LignePanier(this, produit, quantite, produit.getPrix());
            lignes.add(ligne);
        } else {
            ligne.setQuantite(quantiteCible);
        }
        toucher();
        return ligne;
    }
```

Un changement dans une règle (par exemple porter la quantité maximale de 99 à 50) se fait donc dans une constante unique.

5.4 Validation des saisies

La classe `Validation` collecte les erreurs au lieu de s'arrêter à la première. Le formulaire renvoyé à l'utilisateur peut ainsi signaler tous les champs invalides d'un coup. Les contrôles couvrent le texte obligatoire, la longueur, le format de l'e-mail, la complexité du mot de passe (8 caractères minimum, une lettre et un chiffre), le téléphone, la référence produit (mise en majuscules), l'URL d'image (`https` uniquement) et les nombres décimaux ou entiers. L'exception `ValidationException` porte la table des erreurs par champ.

5.5 Requêtes

Toutes les requêtes utilisent des paramètres liés, jamais de concaténation de saisies. Deux détails méritent d'être notés.

— **Recherche par `LIKE`.** Les caractères `%` et `_` saisis par l'utilisateur sont échappés, avec `!` comme caractère d'échappement. Le choix de `!` et non de la barre oblique inverse est volontaire : sous MySQL, la barre oblique inverse a déjà un sens dans les chaînes littérales, ce qui rend l'échappement ambigu.

— **Requêtes dynamiques.** Les vérifications d'unicité sont construites selon les champs fournis, plutôt que d'utiliser `(:param IS NULL OR ...)`, une forme dont le comportement avec des paramètres nuls varie selon le pilote.

5.6 Injection de dépendances avec CDI

5.6.1 Pourquoi CDI sur Tomcat

Tomcat n'est pas un serveur d'applications complet : il ne fournit pas CDI. L'injection par `@Inject` est pourtant la manière propre de relier servlets, services et gestionnaire de transactions sans `new` partout. L'implémentation de référence, Weld, existe en version « servlet » qui s'embarque dans le WAR.

5.6.2 Mise en place

Trois fichiers suffisent.

— `WEB-INF/beans.xml`, en mode `annotated` : seules les classes portant une annotation de portée deviennent des beans, ce qui limite l'analyse.

— `META-INF/context.xml` : déclare à Tomcat la ressource `BeanManager`.

— `WEB-INF/web.xml` : la référence `resource-env-ref` correspondante.

Au démarrage de Tomcat, la ligne `WELD-ENV-001100` (« Tomcat 7+ detected, CDI injection will be available in Servlets, Filters and Listeners ») confirme que l'injection fonctionne.

5.6.3 La fabrique d'EntityManager

La fabrique d'`EntityManager` coûte cher à créer et peut être partagée entre les fils d'exécution ; l'`EntityManager` lui-même ne le peut pas. Un producteur CDI crée donc **une** fabrique pour toute l'application, et un observateur l'initialise dès le démarrage.

Listing 5.3. EntityManagerFactoryProducer

```
@ApplicationScoped
public class EntityManagerFactoryProducer {

    @Produces
    @ApplicationScoped
    public EntityManagerFactory creer() {
        return PersistenceConfig.createEntityManagerFactory();
    }

    public void fermer(@Disposes EntityManagerFactory emf) {
        if (emf.isOpen()) {
            emf.close();
        }
    }

    /**
     * Forces the creation at startup: a wrong password or an unreachable MySQL
```

```
     * shows up in the Tomcat log when the application starts, not at the
     * first visitor's request.
     */
    void demarrer(@Observes @Initialized(ApplicationScoped.class) Object evenement,
                  EntityManagerFactory emf) {
        emf.isOpen();
    }
}
```

L'initialisation dès le démarrage est une décision de confort : un mot de passe erroné ou une base arrêtée apparaît dans la console de Tomcat à l'ouverture de l'application, et non à la requête du premier visiteur. La méthode annotée `@Disposes` ferme la fabrique à l'arrêt, ce qui évite de laisser des connexions ouvertes lors d'un redéploiement.

CHAPITRE 6

Couche web : contrôleurs MVC2

6.1 Le patron MVC2 appliqué

Dans MVC2, le contrôleur est une servlet. Elle reçoit la requête, appelle le modèle (les services), dépose le résultat dans la requête, puis transmet à une JSP qui se contente d'afficher. Les JSP sont placées sous `WEB-INF/views` : le navigateur ne peut donc jamais les demander directement, seule une servlet peut les afficher.

J'ai retenu **une servlet par domaine de gestion**, avec un paramètre `action` qui distingue les opérations (tableau 6.1). Une servlet par action multiplierait les classes ; un contrôleur unique deviendrait trop long.

Table 6.1. Contrôleurs, adresses et actions

Servlet	Adresse	Actions	Accès
`VitrineServlet`	`/vitrine`	GET : `liste`, `detail`	public
`ClientServlet`	`/client`	GET : `connexion`, `inscription` ; POST : `connexion`, `inscription`, `deconnexion`	public
`PanierServlet`	`/panier`	GET : `voir`, `resume` ; POST : `ajouter`, `modifier`, `retirer`, `vider`	client
`ProduitServlet`	`/admin/produits`	GET : `liste`, `nouveau`, `editer` ; POST : `enregistrer`, `basculer`, `supprimer`	admin
`CategorieServlet`	`/admin/categories`	GET : `liste`, `nouveau`, `editer` ; POST : `enregistrer`, `supprimer`	admin

Une règle simple s'applique partout : **une action qui modifie des données passe par POST**, jamais par GET. Un lien ou une image placés sur un autre site ne peuvent ainsi pas déclencher une suppression.

6.2 Classe de base

Les cinq servlets héritent de `BaseServlet`, qui factorise ce qui se répète : lecture de l'action, lecture sécurisée des identifiants et des entiers, affichage d'une vue, redirection, réponse JSON et reconnaissance d'une requête AJAX (en-tête `X-Requested-With`). Une erreur de saisie (identifiant non numérique, par exemple) y est traitée une fois, au lieu d'être dispersée.

6.3 Redirection après POST et messages éphémères

Après un POST réussi, l'application ne renvoie pas directement une page : elle **redirige** vers une page GET. C'est le motif Post/Redirect/Get. Sans lui, un rafraîchissement du navigateur

renverrait le formulaire, et ajouterait une seconde fois le produit au panier.

La redirection fait perdre la requête, donc le message de confirmation (« Produit ajouté ») doit survivre un aller-retour. La classe `Flash` le range en session, et un filtre le transfère à la requête suivante, puis l'efface : le message ne s'affiche qu'une fois.

6.4 Filtres

Deux filtres s'exécutent avant les servlets, dans cet ordre.

1. `SecurityFilter` sur toutes les adresses : encodage UTF-8, en-têtes de sécurité, contrôle du jeton CSRF sur les requêtes POST, transfert du message éphémère.
2. `AuthFilter` sur `/panier` et `/admin/*` : exige un client connecté pour le panier, un administrateur pour l'administration.

L'accès est contrôlé **par adresse, dans un filtre**, et non dans chaque servlet. Une nouvelle page d'administration est protégée dès qu'elle est placée sous `/admin`, sans qu'il faille y penser. Un visiteur non connecté est redirigé vers la connexion, avec mémorisation de la page demandée ; un client qui tente d'ouvrir l'administration reçoit une erreur 403.

6.5 Réponses AJAX

Le mini-panier appelle `PanierServlet` en AJAX. La servlet reconnaît la requête et répond en JSON ; sans AJAX, elle redirige comme n'importe quelle page. Le JSON est écrit à la main, sans bibliothèque, car le contenu est petit : état du panier, nombre d'articles, total déjà formaté, lignes. Les montants arrivent sous forme de texte formaté : le navigateur n'a **aucun calcul d'argent** à faire. Les caractères `<`, `>` et `&` sont échappés dans le JSON, au cas où il serait un jour inséré dans du HTML.

Pour un visiteur non connecté, la réponse est un JSON `connexion requise` avec l'adresse de la page de connexion, que le script suit.

CHAPITRE 7

Vues JSP/JSTL et interface

7.1 Principes

— **Aucun scriptlet.** Le `web.xml` déclare `scripting-invalid` : une balise `<% %>` dans une JSP fait échouer la compilation. Les pages n'utilisent que le langage d'expression (EL) et JSTL, ce qui empêche de glisser de la logique dans la vue.

— **Échappement systématique.** Toute valeur venue de l'utilisateur ou de la base passe par `<c:out>` ou par un gabarit qui l'utilise. C'est la défense principale contre l'injection de script (XSS).

— **Pas de dépendance externe.** jQuery est livré dans l'application, la police et les icônes sont locales (icônes en SVG intégré). L'application fonctionne sans réseau.

— **Fonctionne sans JavaScript.** Chaque action est un vrai formulaire POST ; le script ne fait qu'améliorer le parcours (panier sans rechargement).

7.2 Gabarits (tag files)

Les éléments répétés sont des fichiers de balises dans `WEB-INF/tags`, appelés avec le préfixe `t:`. Une modification de mise en page se fait en un seul endroit (tableau 7.1).

Table 7.1. Gabarits JSP

Balise	Rôle
`t:layout`	Page complète : barre latérale, entête mobile, message éphémère, tiroir du panier, pied de page.
`t:champ`	Champ de formulaire avec étiquette, aide et message d'erreur lié au champ.
`t:csrf`	Champ caché portant le jeton anti-CSRF.
`t:visuel`	Image du produit, ou dégradé coloré avec initiales quand il n'y en a pas.
`t:prix`, `t:stock`	Formatage du prix (« 129,90 MAD ») et étiquette de stock (disponible, dernières unités, épuisé).
`t:icone`, `t:vide`	Icônes SVG et page d'état vide avec illustration.

7.3 Les pages

Dix vues couvrent l'application : liste et fiche produit de la vitrine, connexion, inscription, panier, liste et formulaire des produits, liste et formulaire des catégories, page d'erreur. La page d'erreur sert pour les codes 403, 404, 405 et 500 et n'affiche jamais de trace technique.

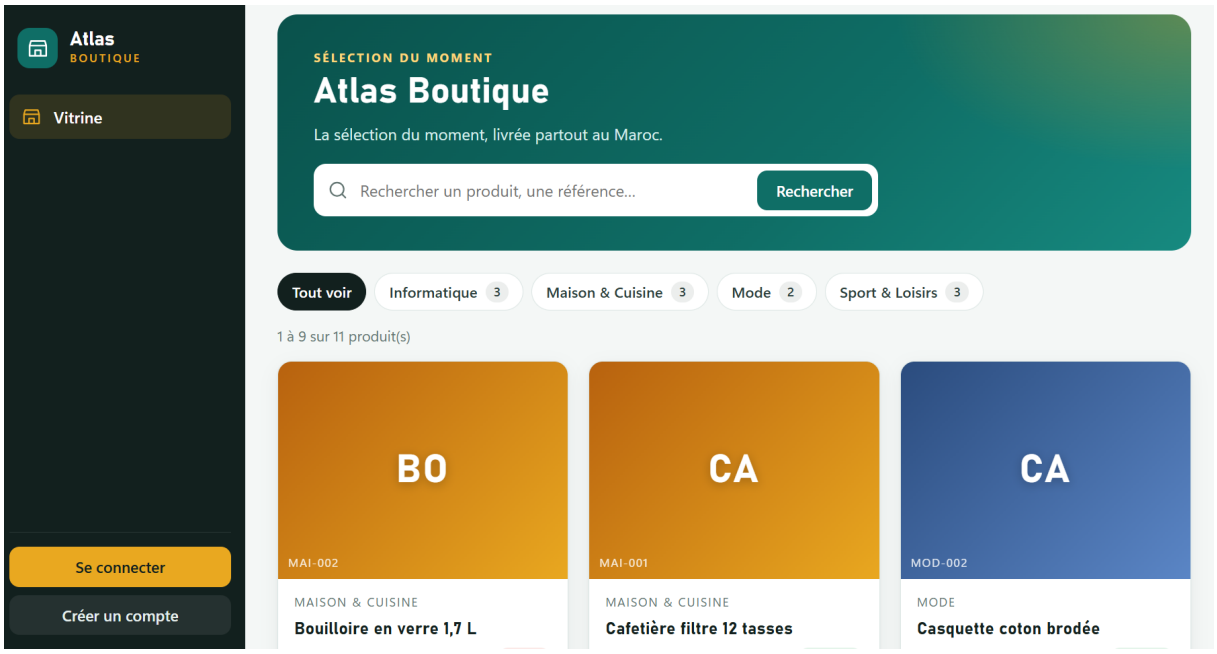


Figure 7.1. La vitrine : filtre par catégorie, recherche et grille de produits

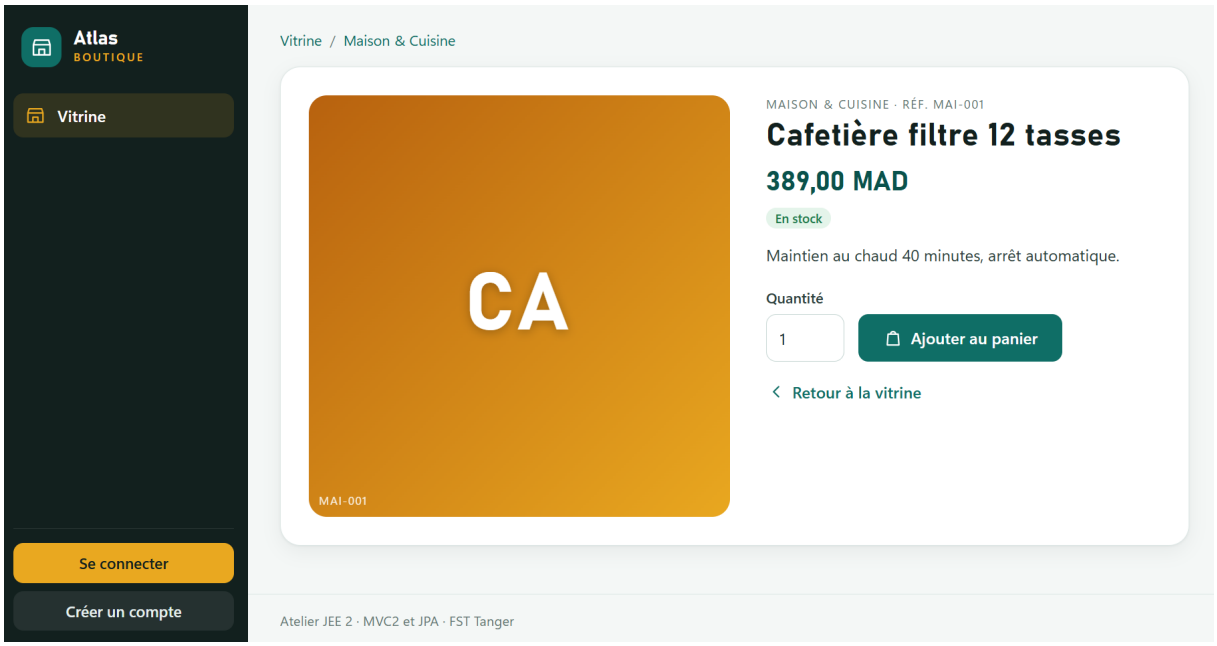


Figure 7.2. Fiche produit avec étiquette de stock et choix de la quantité

7.4 Formulaires et validation

Quand un formulaire contient des erreurs, la page est réaffichée avec les valeurs saisies (sauf les mots de passe) et un message sous chaque champ fautif. Le test se fait côté serveur : les attributs HTML (`required`, `maxlength`) n'améliorent que le confort, puisqu'ils peuvent être contournés.

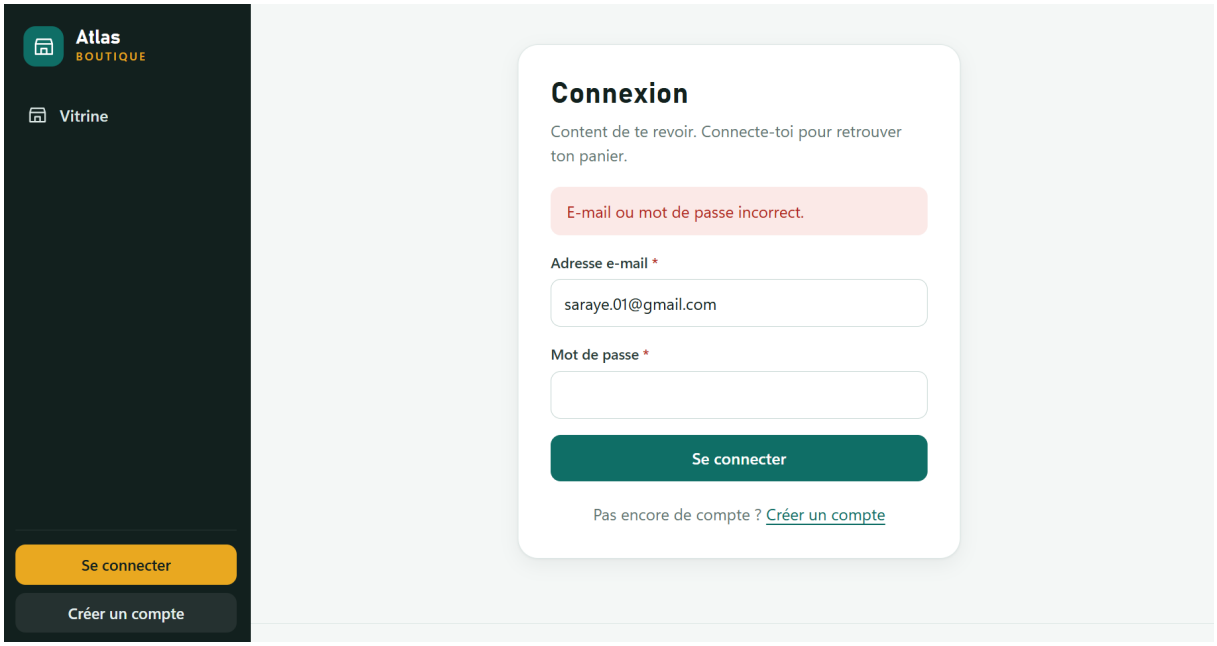


Figure 7.3. Inscription refusée : messages sous les champs invalides

7.5 Mini-panier en jQuery

Un clic sur « Ajouter au panier » envoie le formulaire en AJAX. Le serveur répond par l'état complet du panier ; le script reconstruit le tiroir latéral, met à jour le badge du nombre d'articles et affiche une notification. Les boutons + et − et « Retirer » du tiroir appellent la même servlet.

Deux décisions de sécurité structurent le script.

— Le texte venu du serveur est inséré avec `.text()`, jamais comme HTML : même un nom de produit piégé ne s'exécute pas.

— Le jeton CSRF est lu dans une balise `meta` de la page et envoyé en en-tête `X-CSRF-Token`.

7.6 Administration

Un administrateur voit dans la barre latérale une section dédiée. La liste des produits indique l'état (en vente ou retiré) et propose trois actions : modifier, retirer ou remettre en vente, supprimer. Retirer un produit de la vente le conserve en base et dans les paniers existants ; supprimer est refusé si le produit s'y trouve encore.

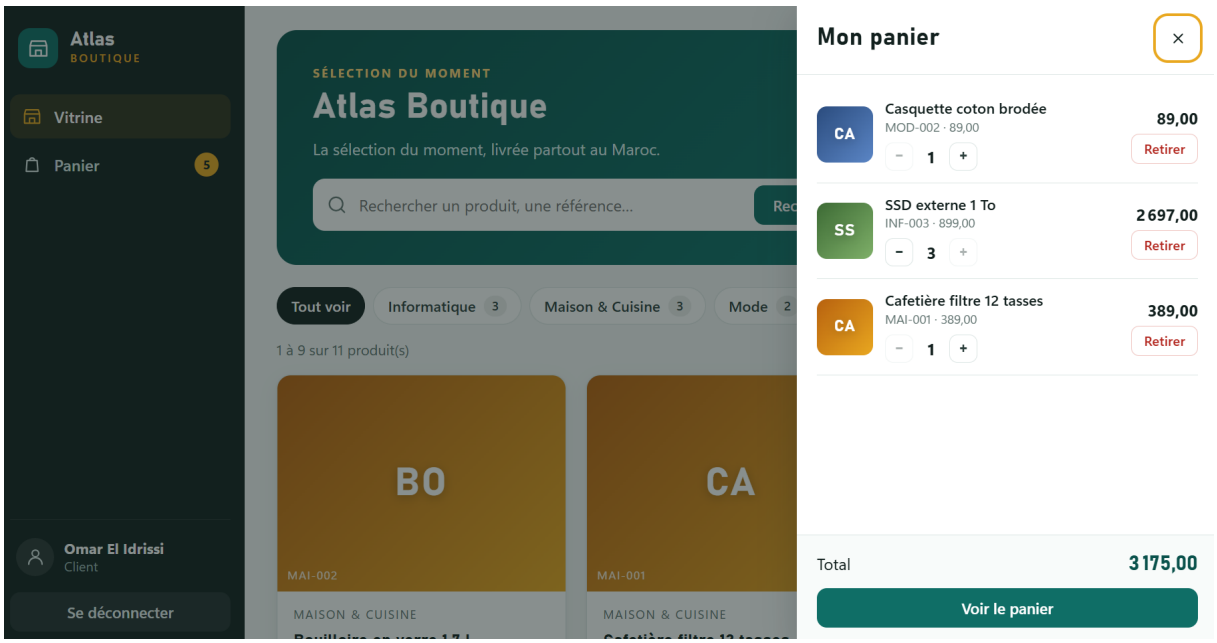


Figure 7.4. Ajout au panier sans rechargement : tiroir latéral, badge et notification

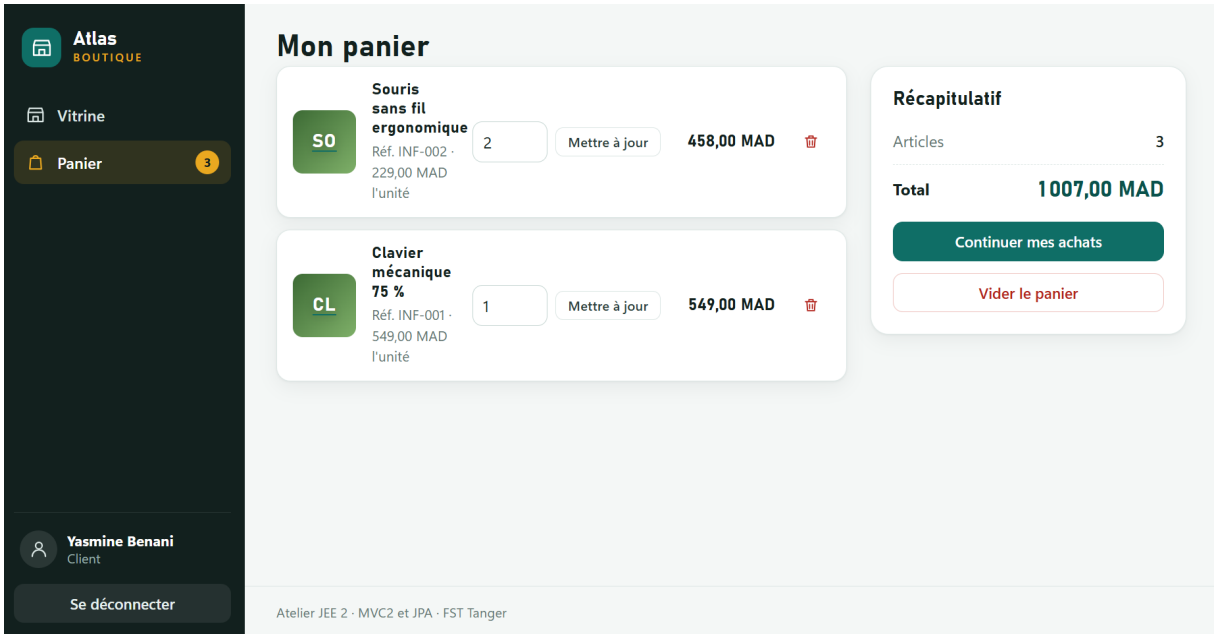


Figure 7.5. Page du panier : lignes, quantités, total et récapitulatif

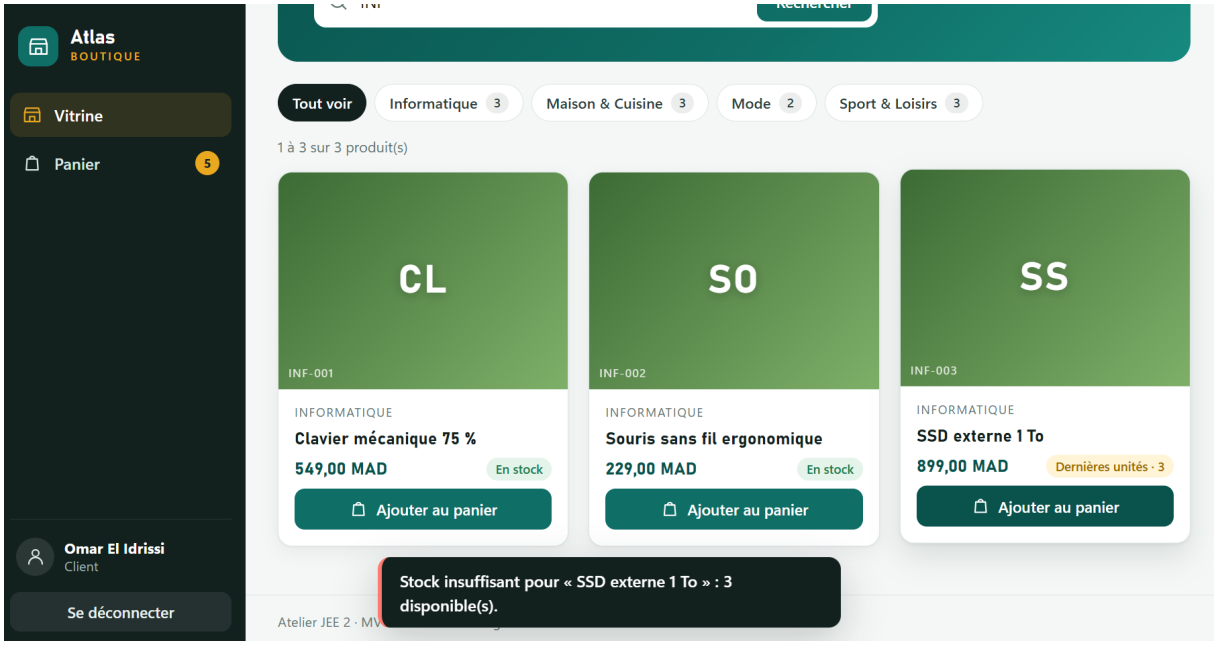


Figure 7.6. Refus d'une quantité supérieure au stock disponible

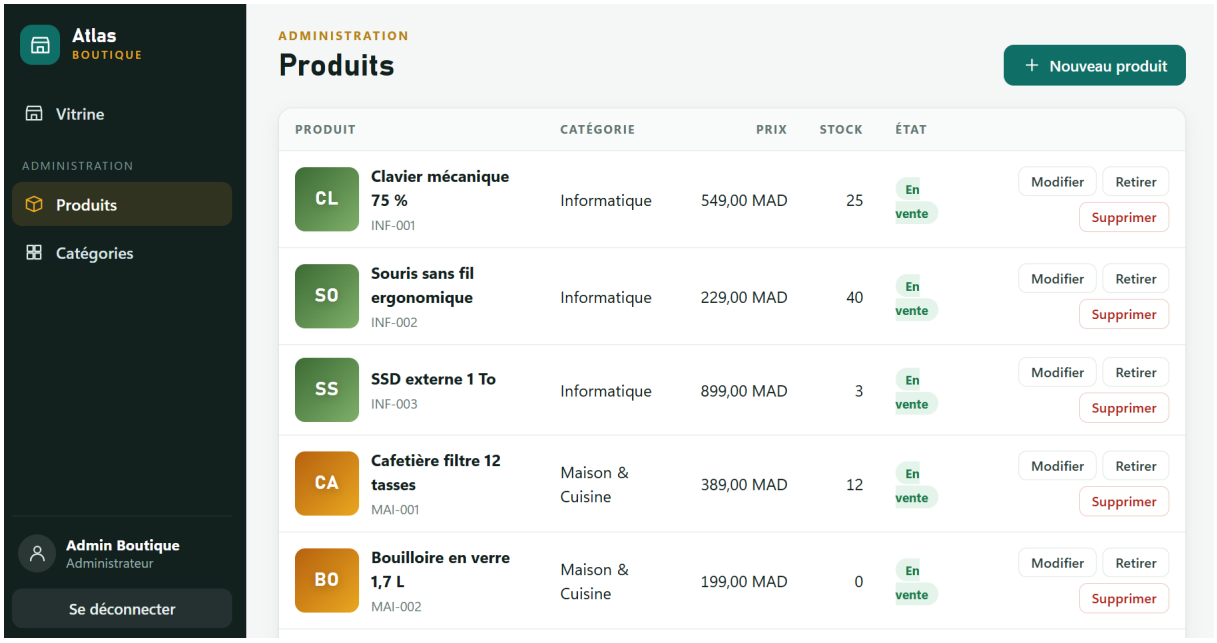


Figure 7.7. Administration : liste des produits et actions

7.7 Adaptation aux petits écrans

En dessous de 860 pixels de large, la barre latérale devient un menu coulissant ouvert par un bouton, la grille de produits passe à une colonne et les lignes du panier se réorganisent. Les animations sont désactivées pour les personnes qui l'ont demandé dans leur système (`prefers-reduced-motion`).



Figure 7.8. Affichage sur petit écran

CHAPITRE 8

Sécurité

La sécurité est intégrée à la structure de l'application. Le tableau 8.1 associe chaque menace courante [7, 8] à la protection mise en place et à l'endroit où elle se trouve.

Table 8.1. Menaces et protections

Menace	Protection	Où
Injection SQL	Requêtes JPQL paramétrées ; saisies de recherche échappées pour `LIKE`	services
XSS	Échappement de toute sortie (`c:out`) ; JSON échappé ; politique CSP sans script en ligne	vues, filtre
CSRF	Jeton de session obligatoire sur tout POST (champ `_csrf` ou en-tête `X-CSRF-Token`)	`SecurityFilter`
Fixation de session	Changement d'identifiant de session et nouveau jeton CSRF à la connexion	`ClientServlet`
Mots de passe volés	Hachage PBKDF2-HMAC-SHA256, sel aléatoire, 600 000 itérations	`PasswordHasher`
Énumération de comptes	Même message pour « e-mail inconnu » et « mauvais mot de passe » ; hachage factice quand le compte n'existe pas	`JpaInternauteService`
Force brute	5 échecs en 15 minutes par couple adresse IP + e-mail, puis blocage temporaire	`LoginThrottle`
Redirection ouverte	Le retour après connexion n'accepte qu'un chemin interne de l'application	`BaseServlet`
Accès non autorisé	Contrôle par adresse dans un filtre (`/panier`, `/admin/*`)	`AuthFilter`
Clickjacking	`X-Frame-Options: DENY` et `frame-ancestors 'none'`	`SecurityFilter`
Fuite d'informations	Pages d'erreur génériques ; pages sensibles en `Cache-Control: no-store`	`web.xml`, filtre

8.1 Jeton CSRF

Un site malveillant peut faire envoyer au navigateur d'un client connecté une requête POST vers l'application, avec son cookie de session. Sans défense, l'action s'exécuterait. Le jeton, propre à la session, est inséré dans chaque formulaire et comparé à celui de la session à l'arrivée du POST ; le site malveillant ne peut pas le connaître. Il est changé à la connexion. La comparaison se fait en temps constant, pour ne rien révéler par la durée du calcul.

8.2 Mots de passe

Un mot de passe n'est jamais stocké. La base contient une empreinte au format `pbkdf2_sha256$itérations$sel$` PBKDF2 [9] est volontairement lent : un attaquant qui volerait la base devrait payer ce coût pour chaque essai. Le nombre d'itérations est écrit dans l'empreinte, ce qui permet de l'augmenter plus tard sans invalider les comptes existants.

8.3 En-têtes et politique CSP

La politique de sécurité du contenu (CSP) limite l'origine des ressources : scripts uniquement depuis l'application, aucun script en ligne, formulaires envoyés uniquement vers l'application, pas d'intégration dans une autre page. C'est la raison pour laquelle le JavaScript est dans un fichier séparé et non dans les pages. Les autres en-têtes sont `X-Content-Type-Options: nosniff` et `Referrer-Policy: same-origin`. Le cookie de session est `HttpOnly` et la session expire après 30 minutes d'inactivité.

8.4 Ce que la sécurité ne couvre pas

Ces limites sont assumées et reprises au chapitre 11 : pas de HTTPS (donc pas d'attribut `Secure` sur le cookie) car l'application tourne en local ; pas de récupération de mot de passe oublié ; le blocage de la force brute est conservé en mémoire et disparaît au redémarrage ; la politique CSP autorise encore les styles en ligne.

CHAPITRE 9

Tests et validation

9.1 Stratégie

Deux niveaux se complètent.

1. **Contrôles automatiques** : une classe `Verifications` lance 112 vérifications Java, sans framework de test, depuis Eclipse (*Run As, Java Application*). Le choix d'écrire de petites classes de contrôle plutôt que d'introduire JUnit garde le projet sans dépendance supplémentaire ; le chapitre 11 indique que JUnit serait la suite logique.
2. **Parcours manuel** dans le navigateur, avec un scénario par règle de gestion (nominal et limite).

9.2 Contrôles automatiques

Le tableau 9.1 indique ce que couvre chaque classe.

Table 9.1. Classes de contrôle

Classe	Contenu
`PanierCheck`	Règles du panier (ajout, quantités 1 à 99, produit inactif, stock, fusion de lignes, prix figé, total) et cohérence des relations entre entités.
`UtilChecks`	Hachage des mots de passe, formatage des montants, limitation des tentatives de connexion (avec horloge simulée).
`ServiceChecks`	Validation des formulaires (e-mail, mot de passe, référence, prix…) et pagination.
`WebChecks`	Redirection de retour sûre (contre la redirection ouverte), jeton CSRF, JSON du mini-panier (échappement).
`JpaChecks`	Traduction des erreurs MySQL en messages métier et échappement des jokers du `LIKE`.

Ces contrôles ne se connectent pas à la base : ils testent la logique pure (entités, validation, hachage, JSON), ce qui les rend rapides et reproductibles sur n'importe quel poste. Ce qui touche réellement MySQL (mapping, cascades, contraintes, verrou optimiste) est vérifié autrement : par l'inspection directe de la base (chapitre 4) et par le parcours manuel ci-dessous. Chaque contrôle affiche une ligne, et le bilan indique le nombre de succès et d'échecs ; un échec termine le programme avec un code d'erreur.

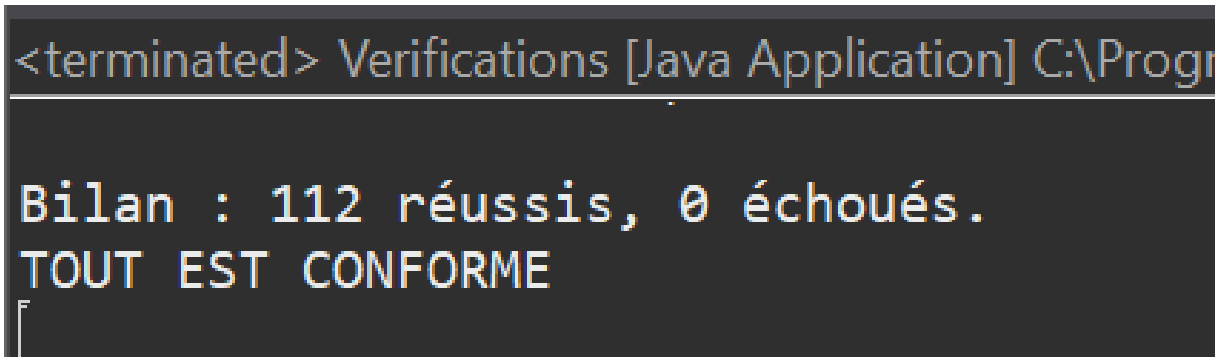


Figure 9.1. Résultat de la classe Verifications

9.3 Parcours manuel

Le tableau 9.2 liste les scénarios exécutés dans le navigateur avec les comptes de démonstration. Les comptes sont `yasmine@example.com` et `omar@example.com` (clients, mot de passe `Client#2026`) et `admin@example.com` (mot de passe `Admin#2026`).

```
PS C:\dev\ecommerce-jpa-mvc2> mysql -u root -p ecommerce_jpa -e "SELECT i.email, p.reference, l.quantite, l.prix_unitair
e FROM ligne_panier l JOIN panier pa ON pa.id=l.panier_id JOIN internaute i ON i.panier_id=pa.id JOIN produit p ON p.id=
l.produit_id ORDER BY i.email;"
Enter password: ****************
+--------------------+-----------+----------+---------------+
| email              | reference | quantite | prix_unitaire |
+--------------------+-----------+----------+---------------+
| omar@example.com   | MOD-002   |        1 |         89.00 |
| omar@example.com   | INF-003   |        3 |        899.00 |
| omar@example.com   | MAI-001   |        1 |        389.00 |
| yasmine@example.com | INF-002   |        2 |        229.00 |
| yasmine@example.com | INF-001   |        1 |        549.00 |
+--------------------+-----------+----------+---------------+
```

Figure 9.2. Contrôle dans MySQL après une action : la ligne de panier correspond à ce qui est affiché

9.4 Ce que les tests ont révélé

La première exécution sur Tomcat a montré que EclipseLink, à la demande de `create-or-extend-tables`, tentait de recréer des contraintes existantes (codes MySQL 1061 et 1826). L'avertissement ne bloquait rien mais montrait un défaut de conception : l'application ne doit pas toucher au schéma. Le réglage `jpa.ddl=none` est donc maintenant celui du fonctionnement normal.

Table 9.2. Scénarios de test manuels

N°	Action	Résultat attendu	Règle
T1	Ouvrir la vitrine, choisir une catégorie, rechercher « clavier »	Liste filtrée, compteur à jour	—
T2	Omar ajoute un produit en stock	Tiroir ouvert, badge à 1, total correct	R4, R5
T3	Omar ajoute le même produit une seconde fois	Une seule ligne, quantité 2	R4
T4	Omar ajoute `MAI-002` (stock 0)	Bouton désactivé ; une requête forcée est refusée	R3
T5	Omar ajoute 4 unités de `INF-003` (stock 3)	Refus avec message de stock	R3
T6	Omar saisit la quantité 0 puis 100	Refus dans les deux cas	R1
T7	Ajouter `MOD-003` (retiré de la vente) par requête forcée	Refus « plus disponible »	R2
T8	Un visiteur non connecté ajoute au panier	Redirection vers la connexion	accès
T9	Inscription avec une adresse déjà utilisée	Message d'erreur sous le champ	R7
T10	Inscription avec un mot de passe « abcdefgh »	Refus (pas de chiffre)	validation
T11	Connexion avec 6 mots de passe faux	Blocage temporaire au 6e essai	force brute
T12	Un client ouvre `/admin/produits`	Page 403	accès
T13	L'administrateur modifie le prix d'un produit présent dans un panier	Le prix de la ligne du panier ne change pas	R5
T14	Supprimer un produit présent dans un panier	Refus, suggestion de le retirer de la vente	R9
T15	Supprimer une catégorie qui contient des produits	Refus	R10
T16	Deux onglets modifient le même produit ; enregistrer les deux	Le second est refusé (conflit de version)	verrou optimiste
T17	Rafraîchir la page après un ajout	Aucun ajout en double (redirection après POST)	PRG
T18	Envoyer un POST sans jeton (outil externe)	Refus 403	CSRF

CHAPITRE 10

Gestion de version

10.1 Principes

Le dépôt `jee-ecommerce-jpa-mvc2` est hébergé sur GitHub. Trois règles ont été suivies.

— **Commits atomiques** : un commit correspond à un changement cohérent (une couche, une fonctionnalité), qui compile seul. L'historique se lit alors comme le déroulement du projet.

— **Messages normalisés** : la convention *Conventional Commits* [10], avec les types `feat`, `fix`, `test`, `docs`, `chore`, et une portée (`persistence`, `service`, `web`, `views`).

— **Rien de secret dans le dépôt** : `db.properties` est dans `.gitignore`, seul le modèle `db.properties.example` est versionné. Avant chaque commit, `git status` permet de vérifier que le fichier n'apparaît pas.

10.2 Historique

Le tableau 10.1 donne la séquence des commits, qui suit l'ordre des étapes.

Table 10.1. Séquence de commits

Étape	Message
Cadrage	`docs: add class diagram and project scope`
Maven	`chore: add Maven project and gitignore`
Persistance	`feat(persistence): add JPA entities, persistence unit and schema tool`
Schéma	`fix(persistence): require a role on every internaute`
Services	`feat(service): add business services, CDI wiring and transaction runner`
Contrôleurs	`feat(web): add MVC2 servlets, filters and security`
Vues	`feat(views): add JSP views, tag files, stylesheet and mini-cart`
Tests	`test: add automated checks and Verifications entry point`
Documentation	`docs: add README and project report`

CHAPITRE 11

Limites et perspectives

11.1 Limites connues

— **Pas de commande.** Le panier ne peut pas être validé (hors périmètre).

— **Pas de HTTPS.** L'application est conçue pour un poste local ; en production, le cookie de session devrait porter l'attribut `Secure` et l'application être servie en HTTPS.

— **Blocage de la force brute en mémoire.** Il est perdu au redémarrage et ne se partage pas entre plusieurs instances du serveur.

— **Tests sans framework.** Les contrôles automatiques sont des programmes Java ordinaires, sans isolation entre tests ni rapport détaillé.

— **Avertissements d'EclipseLink.** Les lignes `[EL Severe]` sur les classes du JDK subsistent (section 3.3) tant que la bibliothèque n'est pas mise à jour.

— **Chargement des relations.** Sans tissage, le panier d'un client est chargé avec lui.

— **Politique CSP.** Elle autorise encore les styles en ligne.

— **Pas de récupération de mot de passe**, ni de vérification de l'adresse e-mail.

11.2 Ajouter la commande

La commande s'ajouterait sans toucher au modèle existant : deux nouvelles entités, `Commande` et `LigneCommande`, relieraient un client à des copies de lignes (produit, quantité, prix figé). La validation serait une seule transaction :

1. relire chaque produit avec verrou optimiste ;
2. vérifier le stock une dernière fois ;
3. décrémenter les stocks ;
4. créer la commande à partir des lignes du panier ;
5. vider le panier.

Si deux clients commandent le dernier exemplaire en même temps, le champ `@Version` du produit fait échouer le second ; le mécanisme est déjà en place et testé sur le panier.

11.3 Autres pistes

— migrer vers la série Jakarta (EclipseLink 4, Tomcat 10 ou 11), ce qui supprime l'incident ASM ;

— introduire JUnit et une base de test en mémoire ;

— ajouter la pagination et le tri dans l'administration ;

— mettre en place l'internationalisation des messages.

CHAPITRE 12

Conclusion

Ce travail livre une application de boutique en ligne complète sur le périmètre choisi : vitrine, comptes, panier et administration du catalogue, avec persistance JPA sur MySQL, architecture MVC2 et injection CDI sur Tomcat.

Trois enseignements se dégagent.

— **Placer chaque règle au bon endroit.** Les règles du panier vivent dans l'entité, les transactions dans un composant unique, l'accès dans un filtre. Chaque règle n'a qu'un point de modification, et la base répète les plus importantes avec ses contraintes.

— **Vérifier chaque couche avant la suivante.** Inspecter MySQL avant d'écrire la première servlet a permis de découvrir la colonne de rôle nullable, et lire les journaux de démarrage a révélé l'incompatibilité entre EclipseLink 2.7.5 et Java 17, puis la configuration de schéma trop active.

— **Intégrer la sécurité à la structure.** Jeton CSRF, échappement, requêtes paramétrées, hachage lent et en-têtes sont des éléments de départ.

La commande, le HTTPS et un véritable cadre de tests restent les prolongements naturels. Le modèle actuel est prêt à les accueillir.

Bibliographie

[1] Oracle / JCP. *JSR 338 : Java Persistence 2.2*. https://jcp.org/en/jsr/detail?id=338

[2] Oracle / JCP. *JSR 369 : Java Servlet 4.0*. https://jcp.org/en/jsr/detail?id=369

[3] Eclipse Foundation. *EclipseLink Documentation (2.7)*. https://www.eclipse.org/eclipselink/documentation/

[4] Red Hat. *Weld, implémentation de référence de CDI*. https://weld.cdi-spec.org/

[5] Apache Software Foundation. *Apache Tomcat 9 Documentation*. https://tomcat.apache.org/tomcat-9.0-doc/

[6] Oracle. *MySQL 8.0 Reference Manual*, section « CHECK Constraints ». https://dev.mysql.com/doc/refman/8.0/en/

[7] OWASP. *Cross-Site Request Forgery Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/

[8] OWASP. *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/

[9] B. Kaliski. *RFC 8018 : PKCS #5 v2.1 (PBKDF2)*. https://www.rfc-editor.org/rfc/rfc8018

[10] *Conventional Commits 1.0.0*. https://www.conventionalcommits.org/

[11] M. Fowler. *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002 (Single Table Inheritance, Front Controller).

CHAPITRE A

Arborescence du projet

```
jee-ecommerce-jpa-mvc2/
  pom.xml
  README.md
  .gitignore
  docs/                       class diagram (Mermaid and TikZ)
  src/main/java/ma/fstt/
    entities/                 Internaute, Client, Administrateur, Vitrine,
                              Categorie, Produit, Panier, LignePanier
    exception/                MetierException, StockInsuffisantException,
                              ValidationException
    util/                     PasswordHasher, Money, LoginThrottle
    persistence/              PersistenceConfig, SchemaTool
    service/                  service interfaces, Validation, forms, PageResultat
    jpa/                      TransactionRunner, SqlErrors, EntityManagerFactoryProducer,
                              Jpa*Service implementations, Requetes
    web/                      BaseServlet, 5 servlets, SecurityFilter, AuthFilter,
                              Flash, SessionUser, PanierJson
  src/main/resources/
    META-INF/persistence.xml
    db/constraints.sql
    db.properties.example     (db.properties is ignored by Git)
  src/main/webapp/
    index.jsp
    META-INF/context.xml
    WEB-INF/web.xml, beans.xml
    WEB-INF/tags/             layout, champ, csrf, visuel, prix, stock, icone, vide
    WEB-INF/views/            vitrine/, client/, panier/, admin/, erreur.jsp
    assets/css/app.css
    assets/js/app.js, vendor/jquery-3.7.1.min.js
  src/test/java/ma/fstt/      Verifications and check classes
```

CHAPITRE B

Comptes et données de démonstration

Table B.1. Comptes de démonstration

Rôle	Adresse	Mot de passe
Administrateur	`admin@example.com`	`Admin#2026`
Client (panier rempli)	`yasmine@example.com`	`Client#2026`
Client (panier vide)	`omar@example.com`	`Client#2026`

Ces comptes n'existent que dans les données de test créées par `SchemaTool` ; ils ne doivent jamais être utilisés tels quels hors d'un poste de développement.